

Section 3: Application Development (30%) - Study Guide

Disclaimer: Official docs.databricks.com page URLs could not be directly confirmed for all topics via automated search — results surfaced mix of direct Databricks pages and third-party summaries. Key concepts below are grounded in current Databricks products (Mosaic AI Agent Framework, MLflow, Foundation Model APIs, Unity AI Gateway, Genie Agents). Verify each topic at docs.databricks.com using the search terms provided.

Terminology Breakdown

- **LangChain:** An open-source Python framework for building applications powered by LLMs. Provides abstractions for chains, agents, retrievers, memory, and output parsers. Databricks integrates natively with LangChain via MLflow (`mlflow.langchain`).
- **LangGraph:** A LangChain extension for building stateful, multi-step agent workflows using directed graphs. Supported natively by Databricks MLflow tracing.
- **Hallucination:** When an LLM confidently generates factually incorrect or fabricated information not grounded in any provided context.
- **Groundedness:** A quality metric measuring whether an LLM's answer is fully supported by the retrieved context (documents), not invented from training data alone.
- **Prompt Template:** A structured wrapper that inserts dynamic variables (e.g., user query, retrieved context) into a fixed instruction string before sending to an LLM.
- **Guardrails:** Mechanisms that intercept and validate LLM inputs/outputs to enforce safety, privacy, and compliance policies. In Databricks, configured via Unity AI Gateway.
- **Unity AI Gateway:** A Databricks service layer sitting in front of model serving endpoints that enforces rate limits, cost budgets, and safety guardrails across all LLM traffic.
- **Foundation Model APIs:** Databricks managed service that exposes state-of-the-art open LLMs (like Llama, Mistral) via an OpenAI-compatible REST API with three modes: Pay-per-token, Provisioned Throughput, and AI Functions.
- **External Models:** Third-party model providers (OpenAI, Anthropic, Google) that can be connected through Databricks Model Serving using a unified OpenAI-compatible API.
- **MLflow Autologging:** A feature (`mlflow.langchain.autolog()`) that automatically captures LLM calls, tool invocations, and chain steps into an MLflow experiment without any manual logging code.
- **MLflow Tracing:** A feature that records every step of an agent's execution — LLM calls, retrieved documents, tool responses — as a structured "trace" with timing, inputs, and outputs.
- **LLM-as-a-Judge:** A technique where a second, highly capable LLM is used to evaluate the quality of outputs from the primary LLM, scoring them on dimensions like relevance, groundedness, and safety.

- **Embedding Model:** An AI model that converts text into a vector (numerical array). The context length of this model determines the maximum number of tokens it can process per chunk.
- **Model Card / Model Metadata:** Documentation accompanying a model that describes its intended use, training data, evaluation benchmarks (e.g., MMLU, HumanEval), limitations, and license.
- **Genie Agents (formerly Genie Spaces):** Databricks natural-language interfaces that allow users to query structured data in Unity Catalog conversationally. Can be integrated as tools within multi-agent systems.
- **Provisioned Throughput:** A dedicated deployment mode in Foundation Model APIs that guarantees token-per-second capacity, required for production-grade and fine-tuned model serving.
- **Pay-per-token:** A serverless Foundation Model APIs mode best for prototyping where you are charged per input/output token consumed.

1. Select LangChain/similar tools for use in a Generative AI application

Concept: Not all AI tasks require the same framework. You must select the right tool based on the complexity and nature of the pipeline.

Databricks Context & Implementation: * **Simple one-shot LLM calls:** Use Databricks Foundation Model APIs directly (Python SDK or REST). No LangChain required. * **RAG chains (document retrieval + LLM):** Use **LangChain** with a `ChatDatabricks` model, a `DatabricksVectorSearch` retriever, and a `PromptTemplate`. Log the chain using `mlflow.langchain.log_model()`. * **Stateful multi-step agents (branching logic):** Use **LangGraph** which allows you to define agents as directed graphs where different "nodes" represent different LLM calls or tool executions and edges define conditional routing. * **Batch inference at scale:** Use Databricks AI Functions (`ai_query()` , `ai_summarize()`) directly in SQL/Spark — no LangChain needed.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Log and deploy LangChain models with MLflow" * "LangGraph agents on Databricks"

2. Qualitatively assess responses to identify common issues such as quality and safety

Concept: Before deploying an application, you must perform human-readable assessment of LLM outputs to detect failure modes that automated metrics miss.

Databricks Context & Implementation: Common issues to identify qualitatively: * **Hallucination:** The model states something confidently that is untrue and not found in any retrieved context. * **Lack of Groundedness:** The answer is plausible but drifts away from or contradicts the provided documents. * **Safety violations:** The model produces harmful, toxic, biased, or inappropriate content. * **Relevance**

failure: The answer is technically correct but does not address the user's actual question. * **Verbosity / Format issues:** The answer is correct but unnecessarily long, or ignores requested output format.

In Databricks, the **MLflow Review App** allows human evaluators (domain experts or SMEs) to score agent responses directly through a UI, producing labeled data used to improve the agent.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow AI Gateway review app" * "Human evaluation Mosaic AI Agent Framework"

3. Select chunking strategy based on model & retrieval evaluation

Concept: After evaluating retrieval performance (see Section 2, Objective 6), you use those metrics to guide your choice of chunking strategy rather than guessing upfront.

Databricks Context & Implementation: * If **Precision@k is low** (many irrelevant chunks retrieved): Your chunks may be too large. Switch to smaller, more focused chunks to make embeddings more specific. * If **Recall@k is low** (relevant chunks not retrieved): Your chunks may be cutting semantic units in half. Switch to semantic or paragraph-based chunking. * If retrieval is accurate but LLM answer quality is poor: The chunks may lack enough context. Apply Parent-Child (hierarchical) chunking to retrieve a larger parent block for the LLM while indexing small child chunks for search.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "RAG evaluation chunking Databricks"

4. Augment a prompt with additional context from a user's input based on key fields, terms, and intents

Concept: Prompt augmentation means enriching the prompt sent to the LLM beyond the user's raw query — by extracting key signals from the input and injecting structured context.

Databricks Context & Implementation: Suppose a user says: *"What is the return policy for premium members?"*

You extract: * **Key field:** `membership_type = "premium"` * **Intent:** `"return policy query"`

Then inject these into your prompt template:

```
[System]: You are a support assistant. The user is a {membership_type} member.  
[Context]: {retrieved_documents}  
[User]: {user_query}
```

This produces a far more targeted LLM call than passing the raw query alone. In LangChain, this is done using `PromptTemplate` or `ChatPromptTemplate` with named input variables.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "PromptTemplate LangChain MLflow Databricks"

5. Create a prompt that adjusts an LLM's response from a baseline to a desired output

Concept: Prompt engineering is the practice of iteratively refining the prompt to shift LLM behavior toward the exact output your application needs.

Databricks Context & Implementation: Common techniques: * **Zero-shot:** Give clear instructions only (no examples). Useful as a starting baseline. * **Few-shot:** Include 2–3 input/output examples in the prompt to "show" the model the expected format. Most effective for structured outputs. * **Chain-of-thought:** Instruct the model to "think step by step" before giving a final answer, which improves accuracy on complex reasoning tasks. * **System role instructions:** Use the system message to strictly define persona, output format, and forbidden behaviors (e.g., "Never respond in a language other than English. Always cite the source document.").

In Databricks, **MLflow Prompt Registry** allows you to version and track prompt templates alongside their evaluation results so you can roll back to higher-performing versions.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Prompt engineering Databricks Foundation Model APIs" * "MLflow Prompt Registry"

6. Implement LLM guardrails to prevent negative outcomes

Concept: Guardrails are safety mechanisms that intercept LLM inputs before they reach the model and outputs before they reach the user, blocking harmful, inappropriate, or policy-violating content.

Databricks Context & Implementation: Databricks implements guardrails at the **Unity AI Gateway** level: * **Managed (built-in) guardrails:** Enable with a single toggle in the Gateway UI. Include: * **Safety filtering:** Blocks hate speech, violence, sexual content. * **PII detection/redaction:** Detects and redacts sensitive personal information (names, credit card numbers, addresses) from both inputs and outputs. * **Jailbreak detection:** Prevents prompt injection attacks that try to override system instructions. * **Custom SQL-based policies:** Write a SQL function that defines organization-specific rules (e.g., block mentions of confidential project codenames). Attach it to the serving endpoint as either an **ON CALL** (input) or **ON RESULT** (output) policy. * **Application-level guardrails:** Can also be implemented in application code using libraries like `guardrails-ai` before the API call is made.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Configure guardrails Unity AI Gateway" * "AI Gateway service policies Databricks"

7. Select the best LLM based on the attributes of the application to be developed

Concept: Different LLMs have different strengths. You must match the model to the use case based on its performance characteristics, cost, latency, and compliance requirements.

Databricks Context & Implementation: Key attributes to consider: * **Task type:** Coding tasks → CodeLlama or Mistral Instruct. Long document summarization → models with large context windows (e.g., 128K tokens). Chat → Llama 3.x Instruct series. * **Latency requirement:** Smaller 7B–13B models are faster; use for real-time chat. Larger 70B+ models are slower; use for batch or complex reasoning. * **Cost:** Pay-per-token for prototyping; Provisioned Throughput for production. * **Compliance (HIPAA, SOC2):** Use Provisioned Throughput endpoints on Databricks, which keep data within your cloud environment and satisfy compliance requirements. * **Fine-tuned model:** If generic models underperform on a specific domain (e.g., medical, legal), use a fine-tuned version on Provisioned Throughput.

Available on Databricks Foundation Model APIs: Llama 3.x, Mixtral, DBRX, and others in the `system.ai` Unity Catalog schema.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Supported models Foundation Model APIs" * "Provisioned throughput Databricks"

8. Select an embedding model context length based on source documents, expected queries, and optimization strategy

Concept: The embedding model's maximum token limit directly determines the largest possible chunk size. You must align your chunk size to the model's limit and optimize based on how documents and queries are structured.

Databricks Context & Implementation: * **Common limits:** Most embedding models max at 512 tokens (e.g., `bge-small-en`). Some newer models support 8,192+ tokens. * **Selection logic:** * Short, precise Q&A over dense factual documents → 256–512 token chunks → standard 512-token embedding model. * Long narrative documents where context spans many paragraphs → use a long-context embedding model (8K tokens) or apply Parent-Child chunking with a short-context model. * **Critical rule:** Always count chunk sizes in tokens using the embedding model's specific tokenizer (not characters). If a chunk exceeds the model's limit, the text is silently truncated, losing data.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Embedding models Foundation Model APIs" * "Vector Search Databricks embedding model configuration"

9. Select a model from a model hub or marketplace for a task based on model metadata/model cards

Concept: Before committing to a model, you must read its model card to ensure it is appropriate for your task — checking its training data, evaluation benchmarks, known limitations, and license.

Databricks Context & Implementation: * **Databricks Marketplace:** A catalog where you can discover models shared by Databricks, partners, and the community. Accessible from the Databricks UI. * **Unity Catalog `system.ai` schema:** Pre-loaded with Databricks-curated foundation models (e.g., Llama 3, Mixtral). You can inspect model metadata directly in Catalog Explorer. * **What to look for in a model card:** * **Intended use / task type:** Is this model designed for chat, code generation, or embeddings? * **Benchmark scores:** MMLU (general knowledge), HumanEval (coding), MT-Bench (chat quality). * **Training data cut-off:** Affects knowledge currency. * **License:** Some models (e.g., Llama 3) have commercial-use terms you must comply with. * **Known limitations / biases:** Critical for safety assessment.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Databricks Marketplace models" * "Unity Catalog `system.ai` schema"

10. Select the best model for a given task based on common metrics generated in experiments

Concept: Rather than selecting a model by intuition, you run controlled MLflow experiments comparing multiple models and pick the winner based on empirical metrics.

Databricks Context & Implementation: * **MLflow Experiments:** Each model evaluation run is logged as an MLflow experiment run with its own set of metrics (e.g., `groundedness`, `relevance`, `latency_ms`, `cost_per_1k_tokens`). * **MLflow Experiment UI:** Provides side-by-side comparison tables and parallel coordinates plots to visually identify the best-performing model configuration. * **Common evaluation metrics for model selection:** * `answer_correctness`: Is the answer factually right? * `groundedness`: Is the answer supported by retrieved context? * `token_count` / `latency`: Cost and speed tradeoff. * `toxicity` / `safety_score`: Ensures content safety. * **Process:** Log each model run with `mlflow.start_run()`, evaluate with `mlflow.evaluate()`, then compare runs in the Experiment UI.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow evaluate generative AI" * "MLflow experiment tracking comparison"

11. Utilize MLflow and Agent Framework for developing agentic systems

Concept: Databricks provides an integrated stack — MLflow + Mosaic AI Agent Framework — that manages the entire agentic development lifecycle: building, tracing, evaluating, and deploying agents.

Databricks Context & Implementation: * **Development:** Build agents using LangChain, LangGraph, or pure Python. Define tools as Unity Catalog Functions. * **Tracing (Observability):** Enable automatic tracing with `mlflow.langchain.autolog()` . This captures every step — LLM call, tool invocation, retrieved document — as a structured MLflow Trace for debugging. * **Evaluation:** Use `mlflow.evaluate()` with built-in GenAI scorers (groundedness, relevance, safety) or custom scorers to benchmark agent performance before deployment. * **Packaging:** Log the agent as an MLflow model using `mlflow.langchain.log_model()` or `mlflow.pyfunc.log_model()` . * **Deployment:** Register the logged model to Unity Catalog and deploy it to a Databricks Model Serving endpoint.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Agent Framework tutorial" * "mlflow.langchain autolog tracing"

12. Compare the evaluation and monitoring phases of the Gen AI application life cycle

Concept: Evaluation and monitoring sound similar but occur at different points in the lifecycle and serve different purposes.

Databricks Context & Implementation:

Dimension	Evaluation (Pre-Deployment)	Monitoring (Post-Deployment)
When	During development, before release	After deployment, on live traffic
Data Used	Curated benchmark datasets with known answers	Real user inputs and outputs from production
Goal	Validate the agent is ready for production	Detect quality drift, errors, and cost overruns in production
Databricks Tool	<code>mlflow.evaluate()</code> , MLflow Experiment UI	Inference Tables, Databricks Lakehouse Monitoring, Agent Monitoring
Feedback	Drives model/prompt selection decisions	Drives retraining, re-prompting, and alert triggers

- **Evaluation:** Answers "Is this agent good enough to ship?"
- **Monitoring:** Answers "Is this agent still performing well after we shipped it?"

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Agent Monitoring" * "Inference tables Databricks Model Serving"

13. Enable multi-agent systems to leverage Genie Agents or conversational API to retrieve data

Concept: Genie Agents are Databricks-native AI interfaces for querying structured data in Unity Catalog using natural language. In a multi-agent system, your orchestrator agent can call a Genie Agent as a specialized tool to answer data-related questions.

Databricks Context & Implementation:

- * **What Genie Agents do:** Accept a natural language question (e.g., "What were total sales by region last quarter?"), generate and execute the appropriate SQL query against registered Unity Catalog tables, and return a result.
- * **Integration approaches:**
 1. **Genie Agents API (REST/SDK):** Directly call the Genie Agent from your supervisor agent code via the Databricks SDK Conversation API. The supervisor sends a user question; the Genie Agent returns data results.
 2. **MCP Server:** Connect a Genie Agent to your multi-agent system as an MCP-compatible tool. Each Genie Agent has a managed MCP URL: `https://<workspace>/api/2.0/mcp/genie/{genie_space_id}`.
- * **Use case example:** A supervisor agent receives "Summarize the top 3 products by revenue this month and explain why." It routes the data retrieval to a Genie Agent (SQL query) and sends the results to a summarization LLM to generate the explanation.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Genie Agents API Databricks" * "Genie Spaces conversational API" * "Multi-agent systems Databricks MCP"